

Seeding a Database : Code-Along (CodeGrade)

Learning Goals

- Initialize a database with sample data
 - Use the **Faker** package to generate random data
-

Key Vocab

- **Seed**: to fill a database with an initial set of data.
-

Introduction

When working with any application involving a database, it's a good idea to populate your database with some realistic sample data when you are working on building new features. Flask-SQLAlchemy, and many other ORMs, refer to the process of adding sample data to the database as "**seeding**" the database. In this lesson, we'll see some of the conventions and built-in features that make it easy to seed data in an Flask-SQLAlchemy application.

Setup

This lesson is a code-along, so fork and clone the repo.

Run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
$ pipenv install
$ pipenv shell
```

Change into the `server` directory and configure the `FLASK_APP` and `FLASK_RUN_PORT` environment variables:

```
$ cd server
$ export FLASK_APP=app.py
$ export FLASK_RUN_PORT=5555
```

Let's create the database `app.db` with an empty `pets` table:

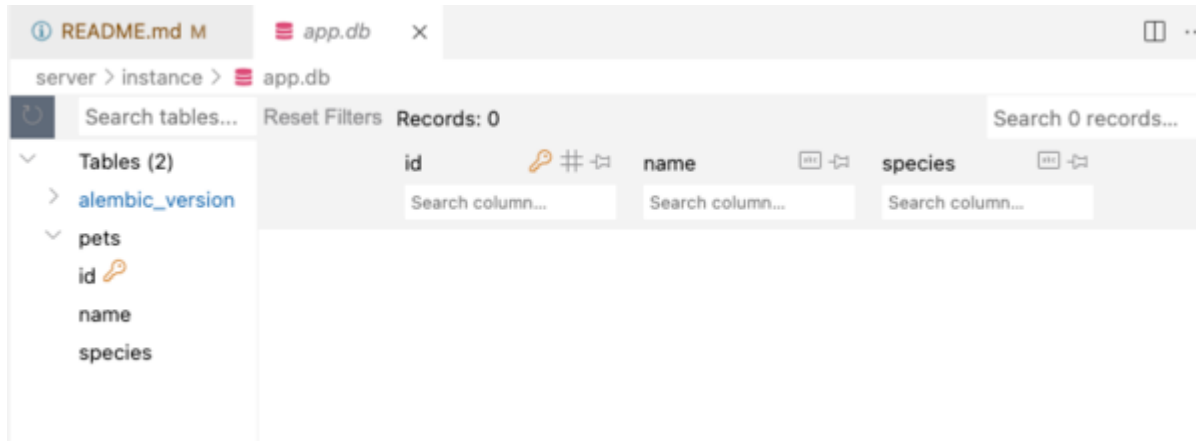
```
$ flask db init

$ flask db migrate -m "Initial migration."

$ flask db upgrade head
```

Confirm the empty `pets` table, either by using the Flask shell or by using a VS Code extension to view the table contents.

```
$ flask shell
>>> Pet.query.all()
[]
>>>
```



NOTE: At any time during the lesson, you can delete the `instance` and `migrations` folders and re-execute the three `flask db` commands (`init`, `migrate`, `upgrade`) to recreate the initial version of the database.

Why Do We Need Seed Data?

In the previous lesson, we used the Flask Shell to call Flask-SQLAlchemy functions to insert rows into the `pets` table. Since the rows are saved in the database rather than in Python's memory, the data persists even after we exit out of the Flask shell.

But how can we share this data with other developers who are working on the same application? How could we recover this data if our development database was deleted? We could include the `app.db` database file in version control, but this is generally considered bad practice. Since our database might get quite large over time, it's not practical to include it in version control (you'll even notice that in our Flask-SQLAlchemy projects' `.gitignore` file, we include a line that instructs Git not to track any `.sqlite3` or `.db` files). There's got to be a better way!

The common approach to this problem is that instead of sharing the actual database file with other developers, we share the instructions for populating data in the database. By convention, the way we do this is by creating a Python file, `seed.py`, which is used to populate our database with sample data.

Take a look at the file `server/seed.py`, which will insert 3 rows into the `pets` table:

```
#!/usr/bin/env python3
#server/seed.py
```

```
from app import app
from models import db, Pet

with app.app_context():

    # Create an empty list
    pets = []

    # Add some Pet instances to the list
    pets.append(Pet(name = "Fido", species = "Dog"))
    pets.append(Pet(name = "Whiskers", species = "Cat"))
    pets.append(Pet(name = "Hermie", species = "Hamster"))

    # Insert each Pet in the list into the database table
    db.session.add_all(pets)

    # Commit the transaction
    db.session.commit()
```

The code creates an application context with the method call `app.app_context()` , and then performs the following within the context:

1. Creates an empty list.
2. Adds several `Pet` instances to the list.
3. Calls `db.session.add_all()` to insert all pets in the list into the table.
4. Calls `db.session.commit()` to commit the transaction.

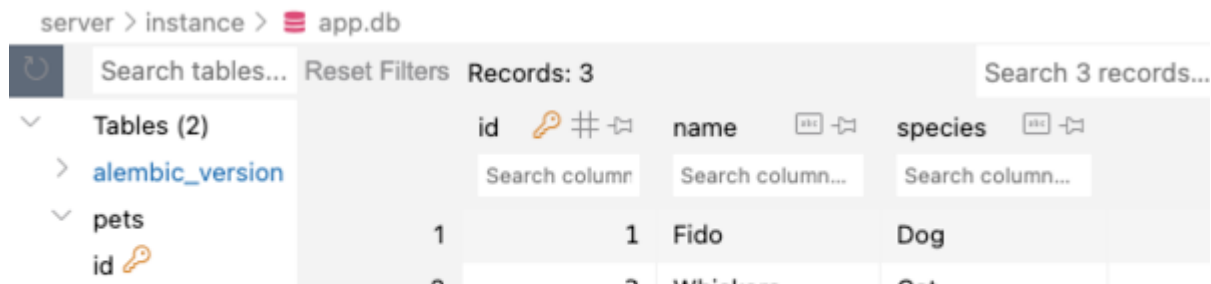
Assuming you are in the `server` directory, type the following to seed the database:

```
$ python seed.py
```

Let's use the Flask shell to query the `pets` table and confirm the 3 pets were added:

```
$ flask shell
>>> from models import db, Pet
>>> Pet.query.all()
[<Pet 1, Fido, Dog>, <Pet 2, Whiskers, Cat>, <Pet 3, Hermie, Hamster>]
```

We can also use the SQLite Viewer (hit the refresh button) to see the new rows:



Let's update `seed.py` to add a fourth pet, a snake named "Slither":

```
#!/usr/bin/env python3
#server/seed.py

from app import app
from models import db, Pet

with app.app_context():

    # Create an empty list
    pets = []

    # Add some Pet instances to the list
    pets.append(Pet(name = "Fido", species = "Dog"))
    pets.append(Pet(name = "Whiskers", species = "Cat"))
```

```
pets.append(Pet(name = "Hermie", species = "Hamster"))
pets.append(Pet(name = "Slither", species = "Snake"))

# Insert each Pet in the list into the database table
db.session.add_all(pets)

# Commit the transaction
db.session.commit()
```

Type `exit()` to exit the Flask Shell and return to the operating system shell. (NOTE: Instead of exiting in and out of the Flask shell, you can open a second terminal and execute the Flask shell and the operating system shell in separate terminals)

```
>>> exit()
$
```

Run `python seed.py` at the command line to reseed the database:

```
$ python seed.py
```

Start the Flask Shell again and query the `pets` table:

```
$ flask shell
>>> Pet.query.all()
[<Pet 1, Fido, Dog>, <Pet 2, Whiskers, Cat>, <Pet 3, Hermie, Hamster>, <Pet 4, Fido, Dog>, <Pet 5, Whiskers, Cat>, <
```

Notice there are 7 rows rather than 4! Each time we run `python seed.py`, we end up adding rows into the existing table. Let's update `seed.py` to delete all rows in the table before adding new rows.

Add the statement `Pet.query.delete()` as the first step in the seeding process:

```
#!/usr/bin/env python3
#server/seed.py

from app import app
from models import db, Pet

with app.app_context():

    # Delete all rows in the "pets" table
    Pet.query.delete()

    # Create an empty list
    pets = []

    # Add some Pet instances to the list
    pets.append(Pet(name = "Fido", species = "Dog"))
    pets.append(Pet(name = "Whiskers", species = "Cat"))
    pets.append(Pet(name = "Hermie", species = "Hamster"))
    pets.append(Pet(name = "Slither", species = "Snake"))

    # Insert each Pet in the list into the database table
    db.session.add_all(pets)

    # Commit the transaction
    db.session.commit()
```

Now if we need to make changes to the pet data and rerun `python seed.py` multiple times, the `pets` table will only contain the 4 rows:

```
>>> exit()
$ python seed.py
```

```
$ flask shell
>>> Pet.query.all()
[<Pet 1, Fido, Dog>, <Pet 2, Whiskers, Cat>, <Pet 3, Hermie, Hamster>, <Pet 4, Slither, Snake>]
>>>
```

NOTE: If you have multiple tables in your database, you'll need to delete first the rows in the tables that have foreign key constraints before deleting the rows in the tables that have the primary key constraints. For example, if you have a `users` table and a `pets` table, and the `pets` table has a foreign key constraint on the `user_id` column, you'll need to delete the rows in the `pets` table before deleting the rows in the `users` table.

Generating Randomized Data

One challenge of seeding a database is thinking up lots of sample data. Ultimately, when you're developing an application, it's helpful to have realistic data, but the actual content is not so important.

One tool that can be used to help generate a lot of realistic randomized data is the [Faker library][faker]. This library is already included in the Pipfile for this application, so we can try it out.

Try out some Faker methods in the Flask shell. First import the package and instantiate a `Faker` instance:

```
$ flask shell
>>> from faker import Faker
>>> fake = Faker()
```

Every time we call the `name()` method, we get a new random name:

```
>>> fake.name()
'Michelle Hill'
>>> fake.name()
'Barbara Harrington'
```


Faker has a lot of random data generator functions that you can use:

```
>>> fake.first_name()
'Eric'
>>> fake.last_name()
'Williams'
>>> fake.email()
'valdezlisa@example.org'
>>> fake.color()
'#c413a3'
>>> fake.address()
'PSC 8907, Box 1499\nAPO AE 66234'
```

Let's update `seed.py` to generate 10 pets, each having a fake first name and a species randomly chosen from a list:

```
#!/usr/bin/env python3
#server/seed.py
from random import choice as rc
from faker import Faker

from app import app
from models import db, Pet

with app.app_context():

    # Create and initialize a faker generator
    fake = Faker()

    # Delete all rows in the "pets" table
    Pet.query.delete()

    # Create an empty list
    pets = []
```

```
species = ['Dog', 'Cat', 'Chicken', 'Hamster', 'Turtle']

# Add some Pet instances to the list
for n in range(10):
    pet = Pet(name=fake.first_name(), species=rc(species))
    pets.append(pet)

# Insert each Pet in the list into the "pets" table
db.session.add_all(pets)

# Commit the transaction
db.session.commit()
```

Rerun the seed script:

```
$ python seed.py
```

Then query the pet table in the Flask shell to view the 10 random pets:

```
$ flask shell
>>> Pet.query.all()
[<Pet 1, Victoria, Dog>, <Pet 2, Michael, Cat>, <Pet 3, Kristie, Chicken>, <Pet 4, Ronald, Hamster>, <Pet 5, Mark, C
>>>
```

Querying the sample data

Let's try out a few queries using `filter_by`. Of course, your results will differ since the data is random.

Filter to get just the cats:

```
>>> Pet.query.filter_by(species = 'Cat').all()
[<Pet 2, Michael, Cat>, <Pet 5, Mark, Cat>, <Pet 6, Shawna, Cat>, <Pet 8, Brian, Cat>, <Pet 10, Elizabeth, Cat>]
```

Filter to get just dogs:

```
>>> Pet.query.filter_by(species = 'Dog').all()
[<Pet 1, Victoria, Dog>]
>
```

Let's count the number of cats:

```
>>> Pet.query.filter_by(species='Cat').count()
5
>>>
```

We can use the `order_by()` function to sort the query result by name:

```
>>> Pet.query.order_by('name').all()
[<Pet 7, Alicia, Hamster>, <Pet 8, Brian, Cat>, <Pet 10, Elizabeth, Cat>, <Pet 9, Jessica, Chicken>, <Pet 3, Kristie, Chicken>]
>>>
```

The `limit()` function restricts the number of rows returned from the query:

```
>>> Pet.query.limit(3).all()
[<Pet 1, Victoria, Dog>, <Pet 2, Michael, Cat>, <Pet 3, Kristie, Chicken>]
```

Conclusion

In this lesson, we learned the importance of having a seed file to quickly set up the database with sample data. We also learned how to use the Faker library to quickly generate randomized seed data.

Resources

- [SQLAlchemy Query Documentation](https://docs.sqlalchemy.org/en/14/orm/query.html)  [\(https://docs.sqlalchemy.org/en/14/orm/query.html\)](https://docs.sqlalchemy.org/en/14/orm/query.html)
- [Faker Documentation](https://faker.readthedocs.io/en/master/)  [\(https://faker.readthedocs.io/en/master/\)](https://faker.readthedocs.io/en/master/)
- [random — Generate pseudo-random numbers - Python](https://docs.python.org/3/library/random.html)  [\(https://docs.python.org/3/library/random.html\)](https://docs.python.org/3/library/random.html)

This tool needs to be loaded in a new browser window

Load Seeding a Database : Code-Along (CodeGrade) in a new window